The actual, overall costs depend on the use case and your policies, so it's not possible to come up with meaningful guidance that is generally applicable. Take, for example, a case where you equip all your developers with access to the distributed tracing solution versus the case where only operations or on-calls get to benefit from it.

Benefits of traces

In a microservices setup, you shouldn't view distributed tracing as optional. You get the dynamic high-level call graph/service map of how your services actually communicate, and you can drill into the request paths, without having to manually (time-stamp-based) correlate service calls via log events.

Popular distributed tracing frontends include the following:

- The Zipkin (https://zipkin.io/) built-in UI
- The Jaeger (https://www.jaegertracing.io/) built-in UI
- Using Grafana with various data source plugins (e.g., Jaeger, Tempo, and X-Ray)
- Various plug-ins for Kibana or OpenSearch dashboards

There's a range of commercial (some SaaS and some OSS) offerings that combine backend/storage and frontend, covering traces among other signal types. We will dive deep into this topic in chapter 6. With this, we've completed the traces cost and benefits discussion. Let's now look at use cases for traces in the context of observability.

2.5.4 Observability with traces

What can we do with traces, and how is what we just saw related to observability? In a developer or QA/test role,

- Developers can use spans to identify slow services and optimize request latencies based on this information.
- QA employees can use traces to troubleshoot a dev/test or preproduction environment to catch performance regressions.
- Developers and testers benefit from distributed traces in debug situations—for example, when a customer reports an issue and live (production) workloads need to be investigated.

In a DevOps or SRE role,

- Based on an alert or a customer reporting an issue, on-calls often find themselves in a first-responder role. They need to be able to search for and identify services or components that could be the cause of the problem.
- For SREs, to assess the overall or component health, trace aggregates (e.g., p90 or p95 values) serve as indicators or input data for recommendations for developers.

With this, we've completed our survey of signal types and use cases. Next, let's move on to some good practices that generally apply.